

PHP and Databases

José Manuel Sánchez Álvarez

Contents

1	PHP and Databases	1
1.1	Introduction to Databases	1
1.2	Database Access from PHP	1
1.2.1	MySQLi and PDO for MySQL	2
1.2.2	CRUD Operations	2
1.2.3	PostgreSQL Connection and CRUD Operations with PDO	4
1.3	CRUD Car Dealership	4
1.3.1	Database Structure	4
1.3.2	HTML Forms for Data Insertion	4
1.3.3	PHP Scripts for CRUD Operations	5
1.3.4	Listing Data	5
1.3.5	Conclusion	6
1.4	Prepared Statements	6
1.4.1	MySQLi and PDO	6
1.4.2	CRUD Operations with Prepared Statements	7
1.4.3	Advantages of Using Prepared Statements	8
1.4.4	Conclusion	8

1 PHP and Databases

Working with databases is a critical aspect of PHP development. In this section, we'll explore how PHP interacts with databases, particularly focusing on MySQL and PostgreSQL.

1.1 Introduction to Databases

A database is an organized collection of data, generally stored and accessed electronically from a computer system. In web development, databases are used to store data such as user information, posts, comments, and more.

1.2 Database Access from PHP

PHP interacts with databases through specific libraries. The most common libraries for MySQL are:

- **MySQLi:** An improved version of the older MySQL library. It provides both procedural and object-oriented interfaces and supports prepared statements, which are crucial for security.
- **PDO (PHP Data Objects):** A database access layer providing a uniform method of access to multiple databases. It doesn't support database-specific syntax, but it's flexible and supports prepared statements.

1.2.1 MySQLi and PDO for MySQL

- Connecting to a MySQL Database

– MySQLi:

```
$mysqli = new mysqli("localhost", "username", "password", "database");

if ($mysqli->connect_error) {
    die("Connection failed: " . $mysqli->connect_error);
}
```

– PDO:

```
try {
    $pdo = new PDO("mysql:host=localhost;dbname=database",
                    "username", "password");

    // Set the PDO error mode to exception
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch(PDOException $e) {
    die("Connection failed: " . $e->getMessage());
}
```

- Listing Databases

– MySQLi:

```
$result = $mysqli->query("SHOW DATABASES");

while ($row = $result->fetch_assoc()) {
    echo $row["Database"] . "\n";
}
```

– PDO:

```
$query = $pdo->query("SHOW DATABASES");

while ($row = $query->fetch(PDO::FETCH_ASSOC)) {
    echo $row["Database"] . "\n";
}
```

1.2.2 CRUD Operations

- Query All Data from a Table

– MySQLi:

```
$result = $mysqli->query("SELECT * FROM table_name");

while ($row = $result->fetch_assoc()) {
    echo $row["column1"] . ", " . $row["column2"] . "\n";
}
```

– **PDO:**

```
$stmt = $pdo->query("SELECT * FROM table_name");

while ($row = $stmt->fetch(PDO::FETCH_ASSOC)) {
    echo $row["column1"] . ", " . $row["column2"] . "\n";
}
```

- More Complex Queries You can perform complex SQL queries such as JOINS, nested SELECTs, etc., using the same methods.

- Insert Data

– **MySQLi:**

```
$sql = "INSERT INTO table_name (column1, column2)
VALUES ('value1', 'value2')";
$mysqli->query($sql);
```

– **PDO:**

```
$sql = "INSERT INTO table_name (column1, column2)
VALUES ('value1', 'value2')";
$pdo->exec($sql);
```

- Delete Rows

– **MySQLi:**

```
$sql = "DELETE FROM table_name WHERE condition";
$mysqli->query($sql);
```

– **PDO:**

```
$sql = "DELETE FROM table_name WHERE condition";
$pdo->exec($sql);
```

- Create Tables

– **MySQLi:**

```
$sql = "CREATE TABLE table_name (id INT, data VARCHAR(100))";
$mysqli->query($sql);
```

– **PDO:**

```
$sql = "CREATE TABLE table_name (id INT, data VARCHAR(100))";
$pdo->exec($sql);
```

1.2.3 PostgreSQL Connection and CRUD Operations with PDO

Connecting to PostgreSQL and performing CRUD operations with PDO is similar to MySQL. The primary difference lies in the DSN (Data Source Name) used in the PDO constructor.

- Connecting to a PostgreSQL Database

```
try {
    $pdo = new PDO("pgsql:host=localhost;dbname=database",
                  1"username", "password");
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch(PDOException $e) {
    die("Connection failed: " . $e->getMessage());
}
```

The CRUD operations for PostgreSQL using PDO are identical to those for MySQL, as PDO abstracts the database details, providing a consistent method for database interaction.

1.3 CRUD Car Dealership

Creating a complete set of CRUD operations for a car dealership database involves several components, including database design, PHP scripts for handling operations, and HTML forms for user input. Let's outline a basic structure for such an application.

1.3.1 Database Structure

1. Tables:

- Car: id, model, price, brand_id, seller_id
- CarBrand: id, name
- Seller: id, name, cars_sold
- Client: id, name
- Purchase: id, client_id, car_id, date

2. Relationships:

- A Car belongs to a CarBrand and a Seller.
- A Purchase links a Client to a Car.

1.3.2 HTML Forms for Data Insertion

1. Add a New Car:

```
<form action="add_car.php" method="post">
    Model: <input type="text" name="model"><br>
    Price: <input type="text" name="price"><br>
    Brand: <select name="brand_id">
        <!-- Options populated from CarBrand table -->
```

```

</select><br>
Seller: <select name="seller_id">
    <!-- Options populated from Seller table -->
</select><br>
<input type="submit" value="Add Car">
</form>

```

2. Add a New Client:

```

<form action="add_client.php" method="post">
    Name: <input type="text" name="name"><br>
    <input type="submit" value="Add Client">
</form>

```

1.3.3 PHP Scripts for CRUD Operations

1. Add a New Car (add_car.php):

```

<? =
// Assume PDO $pdo is already connected to the database
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $stmt = $pdo->prepare(
        "INSERT INTO Car (model, price, brand_id, seller_id)
        VALUES (:model, :price, :brand_id, :seller_id)");
    $stmt->execute([
        ':model' => $_POST['model'],
        ':price' => $_POST['price'],
        ':brand_id' => $_POST['brand_id'],
        ':seller_id' => $_POST['seller_id']
    ]);
    echo "Car added successfully.";
}

```

2. Add a New Client (add_client.php):

```

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $stmt = $pdo->prepare("INSERT INTO Client (name) VALUES (:name)");
    $stmt->execute([':name' => $_POST['name']]);
    echo "Client added successfully.";
}

```

1.3.4 Listing Data

1. List All Cars (list_cars.php):

```

$stmt = $pdo->query("SELECT Car.model, Car.price, CarBrand.name AS brand, Sel
while ($row = $stmt->fetch(PDO::FETCH_ASSOC)) {
    echo "Model: " . $row['model'] . ", Price: " . $row['price'] . ", Brand:
}

```

2. List Cars Bought by Clients (cars_bought_by_clients.php):

```

$stmt = $pdo->query("SELECT Client.name AS client, Car.model FROM Purchase JO
while ($row = $stmt->fetch(PDO::FETCH_ASSOC)) {
    echo "Client: " . $row['client'] . ", Car Model: " . $row['model'] . "<br
}

```

3. List Cars Sold by Sellers (cars_selled_by_sellers.php):

```

$stmt = $pdo->query("SELECT Seller.name AS seller, COUNT(Car.id) AS cars_sell
while ($row = $stmt->fetch(PDO::FETCH_ASSOC)) {
    echo "Seller: " . $row['seller'] . ", Cars Sold: " . $row['cars_selled'
}

```

1.3.5 Conclusion

This setup provides a basic structure for a car dealership application with CRUD operations. The PHP scripts and HTML forms can be expanded and customized as needed. Security considerations, like input validation and prepared statements, have been included to prevent SQL injection. For a real-world application, you'd also need to handle

This overview provides a foundation for working with MySQL and PostgreSQL databases in

PHP. Each of these operations is fundamental for creating dynamic, data-driven web applications. Understanding how to connect to a database, perform CRUD operations, and manage different database engines is crucial for any PHP developer.

1.4 Prepared Statements

Using prepared statements for querying databases is an essential practice in PHP to ensure security, particularly to prevent SQL injection attacks. Prepared statements separate SQL commands from the data, allowing the database to recognize and treat data as such, rather than as part of an SQL command.

1.4.1 MySQLi and PDO

PHP offers two main methods to interact with databases using prepared statements: MySQLi (MySQL Improved) and PDO (PHP Data Objects). Both support prepared statements, but PDO is more flexible as it can interface with various database systems.

- **MySQLi Prepared Statement Connecting to the Database:**

```

$mysqli = new mysqli("localhost", "username", "password", "database");
if ($mysqli->connect_error) {
    die("Connection failed: " . $mysqli->connect_error);
}

```

Prepared Statement:

```

// Prepare a statement
$stmt = $mysqli->prepare("SELECT * FROM table WHERE column = ?");
$stmt->bind_param("s", $param);

```

```
// Set parameters and execute
$params = "value";
$stmt->execute();

$result = $stmt->get_result();
while ($row = $result->fetch_assoc()) {
    // process $row
}

$stmt->close();
```

- **PDO Prepared Statement Connecting to the Database:**

```
try {
    $pdo = new PDO("mysql:host=localhost;dbname=database", "username", "password");
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch(PDOException $e) {
    die("Connection failed: " . $e->getMessage());
}
```

Prepared Statement:

```
$stmt = $pdo->prepare("SELECT * FROM table WHERE column = :param");
$stmt->bindParam(':param', $param, PDO::PARAM_STR);

$params = "value";
$stmt->execute();

while ($row = $stmt->fetch(PDO::FETCH_ASSOC)) {
    // process $row
}
```

1.4.2 CRUD Operations with Prepared Statements

- **Insert Data MySQLi:**

```
$stmt = $mysqli->prepare("INSERT INTO table (column1, column2) VALUES (?, ?)");
$stmt->bind_param("ss", $column1, $column2);

$column1 = "data1";
$column2 = "data2";
$stmt->execute();
```

PDO:

```
$stmt = $pdo->prepare("INSERT INTO table (column1, column2) VALUES (:column1, :column2)");
$stmt->bindParam(':column1', $column1);
$stmt->bindParam(':column2', $column2);

$column1 = "data1";
```

```
$column2 = "data2";
$stmt->execute();
```

- **Update Data MySQLi:**

```
$stmt = $mysqli->prepare("UPDATE table SET column1 = ? WHERE column2 = ?");
$stmt->bind_param("ss", $column1, $column2);

$column1 = "newdata";
$column2 = "condition";
$stmt->execute();
```

PDO:

```
$stmt = $pdo->prepare("UPDATE table SET column1 = :column1 WHERE column2 = :column2");
$stmt->bindParam(':column1', $column1);
$stmt->bindParam(':column2', $column2);

$column1 = "newdata";
$column2 = "condition";
$stmt->execute();
```

- **Delete Data MySQLi:**

```
$stmt = $mysqli->prepare("DELETE FROM table WHERE column = ?");
$stmt->bind_param("s", $param);

$param = "value";
$stmt->execute();
```

PDO:

```
$stmt = $pdo->prepare("DELETE FROM table WHERE column = :param");
$stmt->bindParam(':param', $param);

$param = "value";
$stmt->execute();
```

1.4.3 Advantages of Using Prepared Statements

1. **Security:** They prevent SQL injection attacks.
2. **Performance:** They can improve performance, as the SQL statement template is compiled once and can be reused with different parameters.
3. **Flexibility:** With PDO, you can switch between different database systems without changing the PHP code for preparing and executing the SQL.

1.4.4 Conclusion

Using prepared statements for database queries is a best practice in PHP development. It enhances the security of your applications and can improve performance. Whether using MySQLi or PDO, the concept remains the same: separate SQL structure from the data.